User Manual

Overview

What Is Tier0?

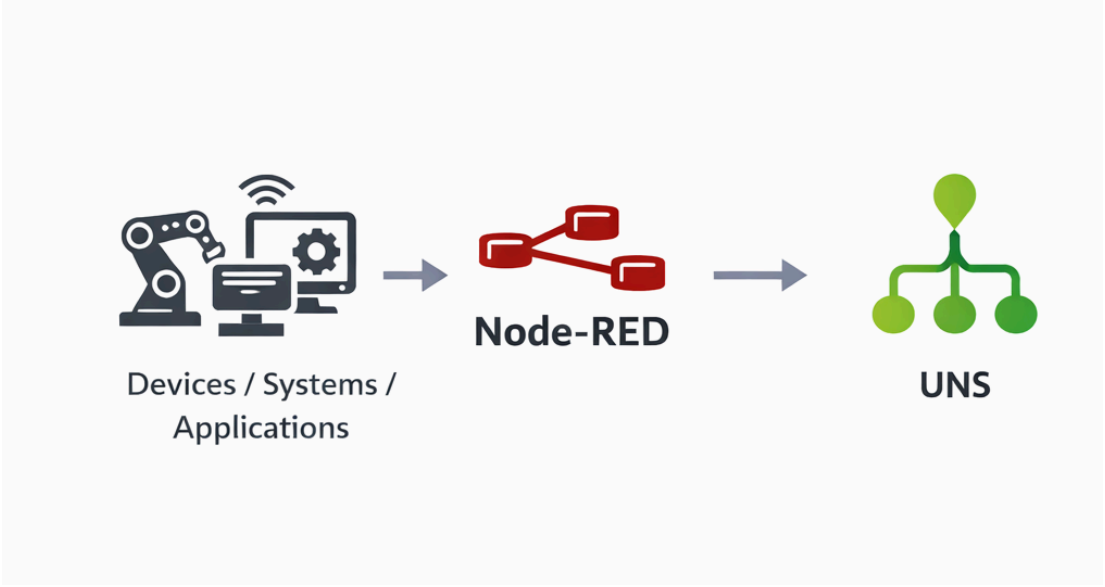
Tier0 is a Unified Namespace (UNS)-based industrial data platform.

It connects data through Node-RED, and organizes it into a unified structure.

How It Works?

Tier0 structures and processes data in a single workflow:

- Node-RED connects and processes data from devices and systems
- Data is published into the Unified Namespace (UNS), structured and stored in Tier0 database

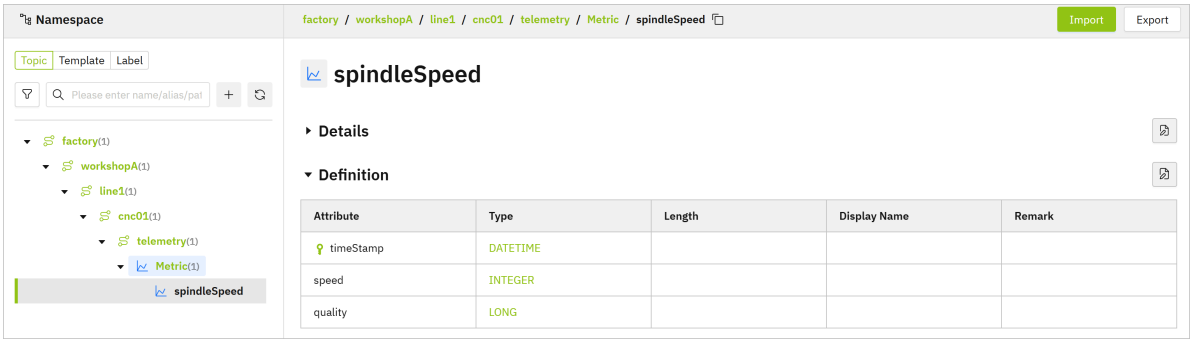


Key Components

Unified Namespace (UNS)

Defines a hierarchical structure for all industrial data.

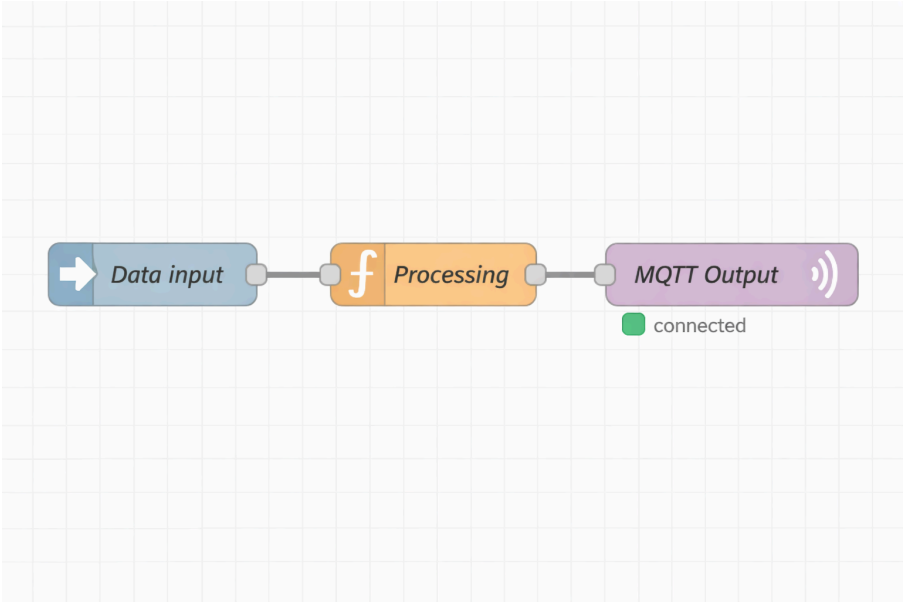
`factory/workshopA/line1/cnc01/telemetry/spindleSpeed`



Node-RED Integration

The core engine for data connection and processing.

- Connect devices, applications, and systems
- Format and publish data into the namespace



Feature List

This section lists the features of Tier0, and each feature has a corresponding guide in the documentation. You can click the links to learn how to use each feature.

Feature	Description	Remarks
Namespace	Models industrial data into a hierarchical structure (folder-file) for clarity	• Private deployment. • For details on the specifications, see System Requirements.
Source Flow	Connect various data sources via drag-and-drop using Node-RED	

Quick Start Guide

Deploy Tier0

System Requirements

Configuration Type	CPU	Memory	HDD/IOPS
Basic Configuration	4 cores	8 GB	100 GB, 2000 IOPS (30% write)
Recommended Configuration	8 cores	16 GB	1 TB, 2000 IOPS (30% write)

Installing Tier0

Linux

1. Make sure you have the following system and components ready.

- Ubuntu Server 24.04
- Docker

Component	Version
Docker Engine - Community	27.4.0
Docker Buildx	v0.19.2
Docker Compose	v2.31.0
Containerd	1.7.24

> ⓘ **note**
>
> Above system and components have been tested. You can try on other versions if you are interested.

2. Clone the project from Github.

```
git clone https://github.com/FREEZONEX/Tier0-Edge.git
```

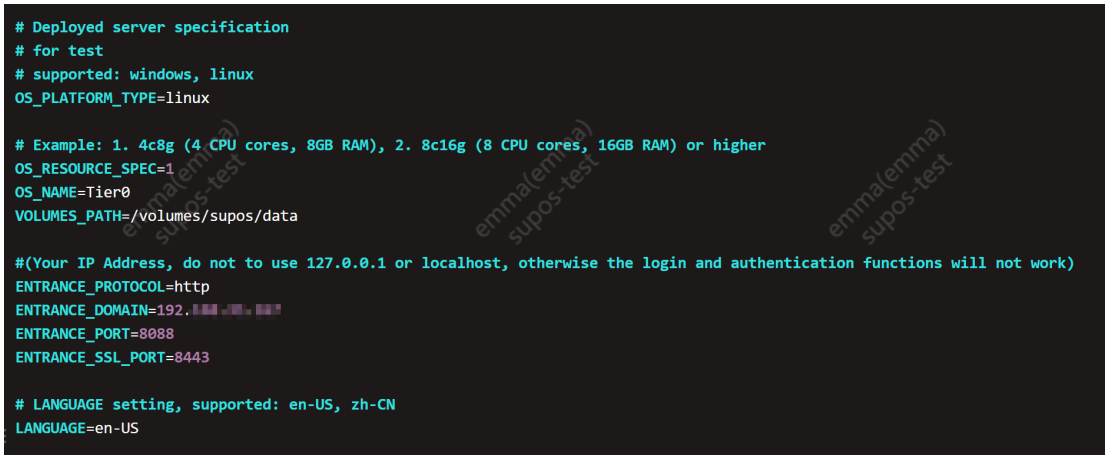
3. Go to the folder **Tier0-Edge/deploy** inside the project.

```
cd Tier0-Edge/deploy
```

4. Enter the **.env.default** file and press the **i** key to edit configuration items as needed.

```
vi .env.default
```

Item	Description
VOLUMES_PATH	The storage path for project data.
ENTRANCE_DOMAIN	Tier0 access address. Normally the IP or domain of the server where you install Tier0. ⚠ **warning** Do not use 127.0.0.1 or localhost, otherwise login and authentication functions **will NOT** work.
LANGUAGE	Tier0 display language. Chinese (zh-CN) and English (en-US) are supported.



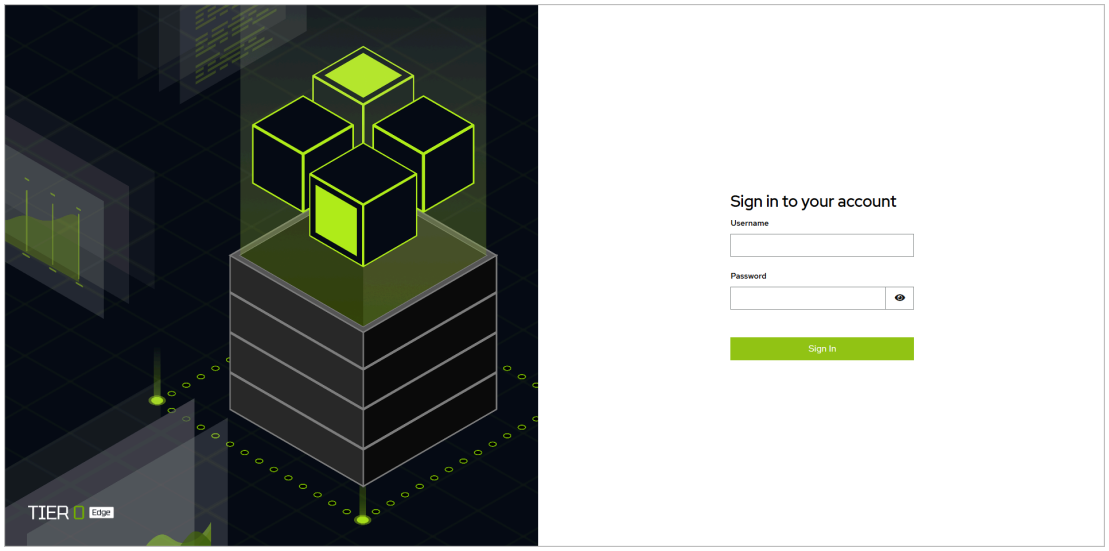
5. Press **ESC** to stop editting, save the file and install Tier0.

```
:wq!
bash bin/install.sh
```

6. Select whether to keep the entered entrance address and whether to use default setup, and then wait for the installation to complete.

```
Available options for ENTRANCE_DOMAIN:
0). Keep current: office.unibutton.com (default)
1). 192.168.31.167
2). 172.17.0.1
3). 172.18.0.1
Select option (0-3), or press Enter to keep current: 0
Keeping current ENTRANCE_DOMAIN: office.unibutton.com
Docker and Docker Compose meet the required versions.
Do you need to customize which services to install? [1] No, use defaults(default) [2] Yes 1
Info: success to generate /root/supOS-CE/bin/init/../../mount/postgresql/init-scripts/create_keycloak_database.sql
Info: success to generate kong_config.yml (auth enabled = true)
Info: loading npm cache complete.
golioth-websocket-datasource/
golioth-websocket-datasource/CHANGELOG.md
golioth-websocket-datasource/gpx_websocket_darwin_amd64
golioth-websocket-datasource/gpx_websocket_darwin_arm64
golioth-websocket-datasource/gpx_websocket_linux_amd64
golioth-websocket-datasource/gpx_websocket_linux_arm
golioth-websocket-datasource/gpx_websocket_linux_arm64
golioth-websocket-datasource/gpx_websocket_windows_amd64.exe
golioth-websocket-datasource/img/
golioth-websocket-datasource/img/assets/
golioth-websocket-datasource/img/assets/golioth-grafana-websockets-plugin-datasource.png
golioth-websocket-datasource/img/assets/grafana-websockets-graphing.png
golioth-websocket-datasource/img/assets/grafana-websockets-plugin-streaming.png
golioth-websocket-datasource/img/logo.svg
golioth-websocket-datasource/LICENSE
golioth-websocket-datasource/MANIFEST.txt
golioth-websocket-datasource/module.js
golioth-websocket-datasource/module.js.LICENSE.txt
golioth-websocket-datasource/module.js.map
```

7. Access Tier0 on a browser and log in.



Windows

> ⓘ **info**
>
> - Make sure you have installed the lastest **Docker Desktop** and **Git** on Windows 10 or 11.
> - Docker must be started.

1. Clone the project from Github in **Git Bash**.

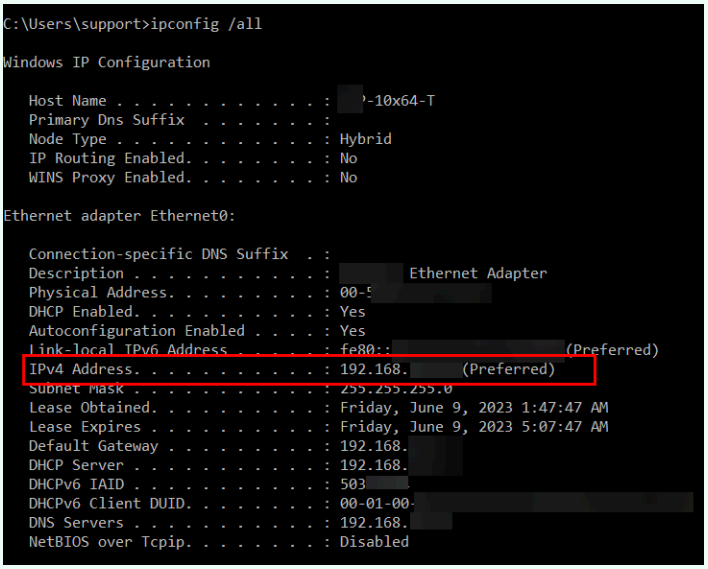
```
git clone https://github.com/FREEZONEX/Tier0-Edge.git
```

2. Go to the folder **Tier0-Edge/deploy** inside the project.

```
cd Tier0-Edge/deploy
```

3. Edit the following attributes in file **.env.default**.

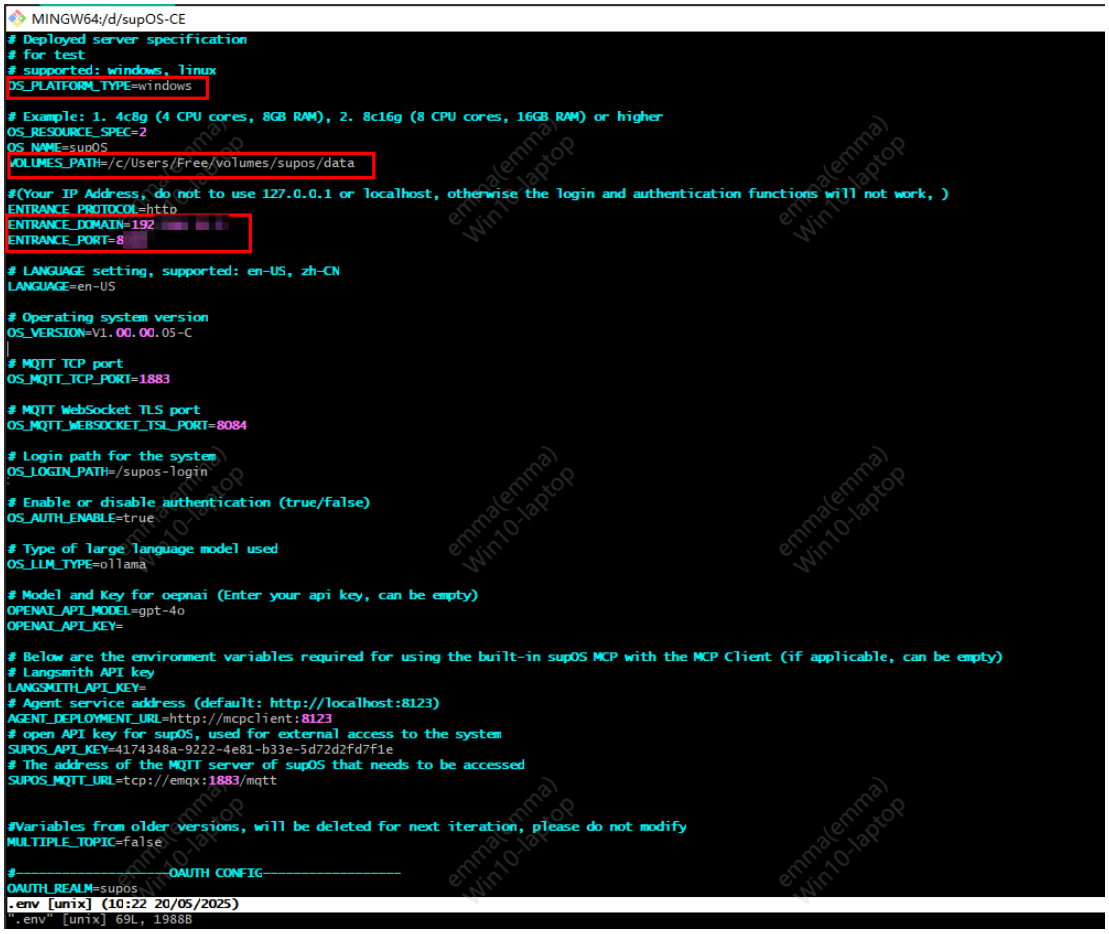
> ✓ **tip**
>
> Before editing .env.default file, execute `ipconfig` in Git Bash to get the IP of the server.



- **OS_PLATFORM_TYPE**: The operating system where Tier0 is installed. Change it to **windows**
- **VOLUMES_PATH**: The storage path for project data.
- **ENTRANCE_DOMAIN/ENTRANCE_PORT**: Tier0 access address. Normally the IP or domain of the server where you install Tier0, and make sure the port is not occupied.

```
vi .env.default
```

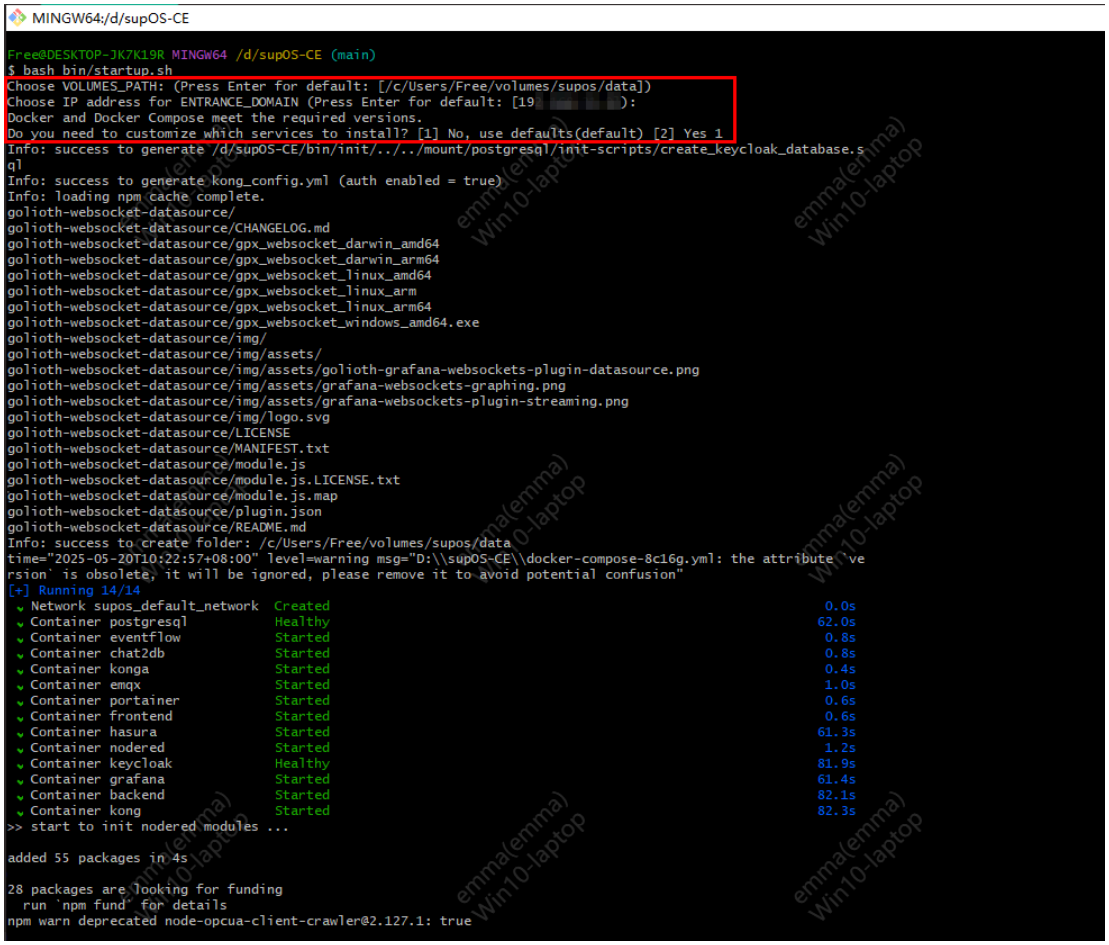
> ⓘ **note**
>
> **.env.default** file is hidden, directly enter the command to open it.



4. Save the file and start installing Tier0.

```
:wq!
bash bin/install.sh
```

5. Select the default configurations and wait for the installation to complete.



6. Access Tier0 on a browser and log in.

Uninstalling Tier0

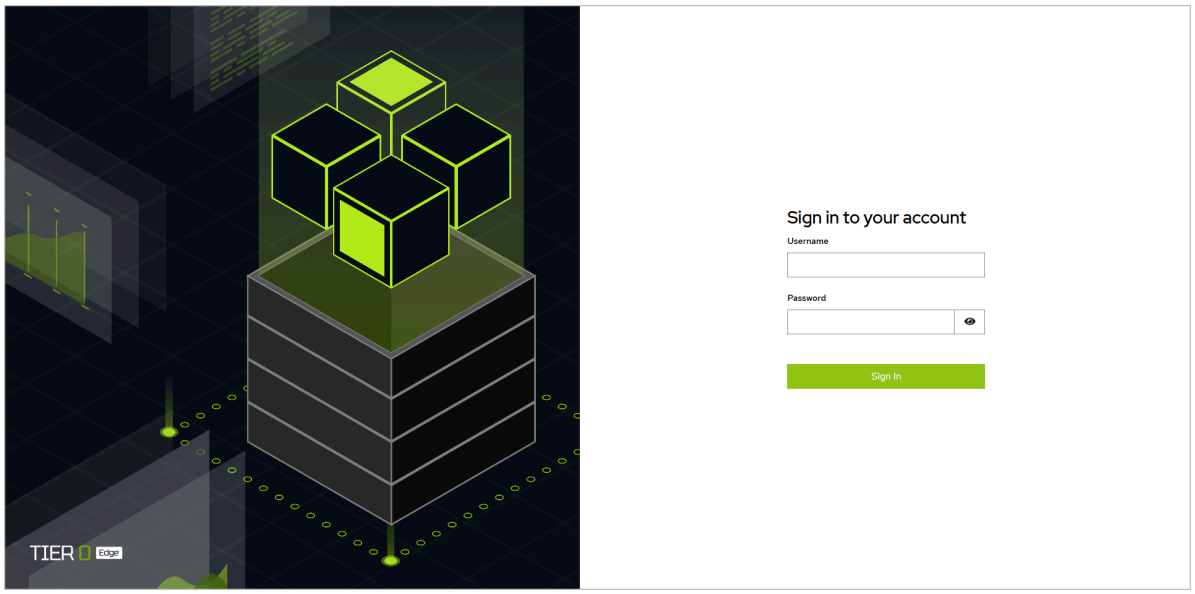
1. Access the path **Tier0-Edge/deploy/bin**.
2. Execute the script **uninstall.sh**.

```
bash uninstall.sh
```

> ⓘ **info**
>
> - **clean-all.sh** will wipe out all project data and uninstall Tier0.
> - Images will not be deleted.
> - If you want to install Tier0 again, make sure you have pulled all images with the newest versions (based on image tags).

Login

1. Access Tier0 with the set domain and port on a browser.
2. On the login page, enter username **tier0** and password **tier0**.



3. Click **Sign In**.

How-to Guide

Build Data Models

Tier0 uses UNS (Unified Namespace) to organize data into a clear tree structure. Each path and topic in the model automatically generates an MQTT topic, making it easy to get real-time data.

Additional Parameter Definition

- **Path-Additional Settings**

Item	Description
Extended Attribute	Add extended attributes for the path. Eg. unit.
Template	Select a template to inherit attributes from it. See Building Template.
Generate Template	Click ⊕ to add an attribute to the path, at the same time you can select to generate a same template for future use.

- **Topic-Topic Type**

Type	Description
State	Works as an API to receive system state data.
Action	Works as an API to execute CRUD commands.
Metric	Real-time data. For example, equipment temperature.

- **Topic-Attribute Generation Method**

Attribute Generation Method	Description
Pre-defined	Customize attributes.
Template	Inherit attributes from the selected template.
Auto-Parsed	Use JSON text to generate attributes with the same structure.

- **Topic-Linked Operation**

Item	Description
Enable History	Enable historical data storage for the topic.

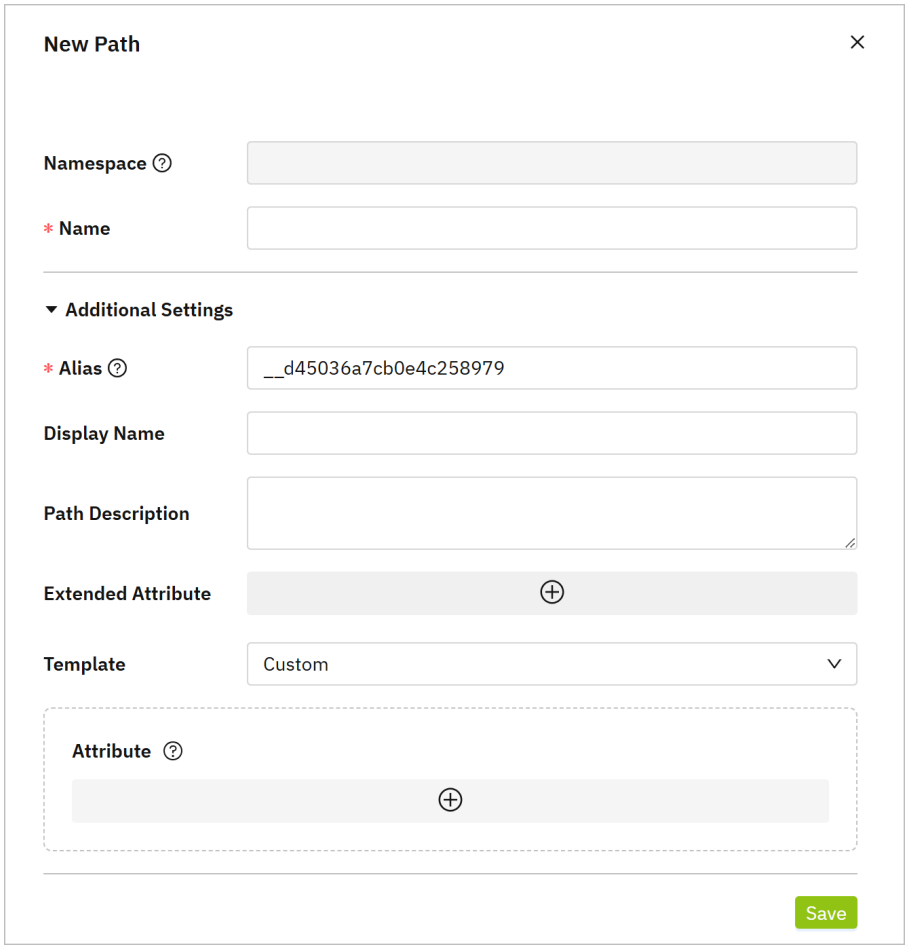
How to Build a Data Model

Based on simple folder-file structure, you can define the data hierarchy to a tree map.

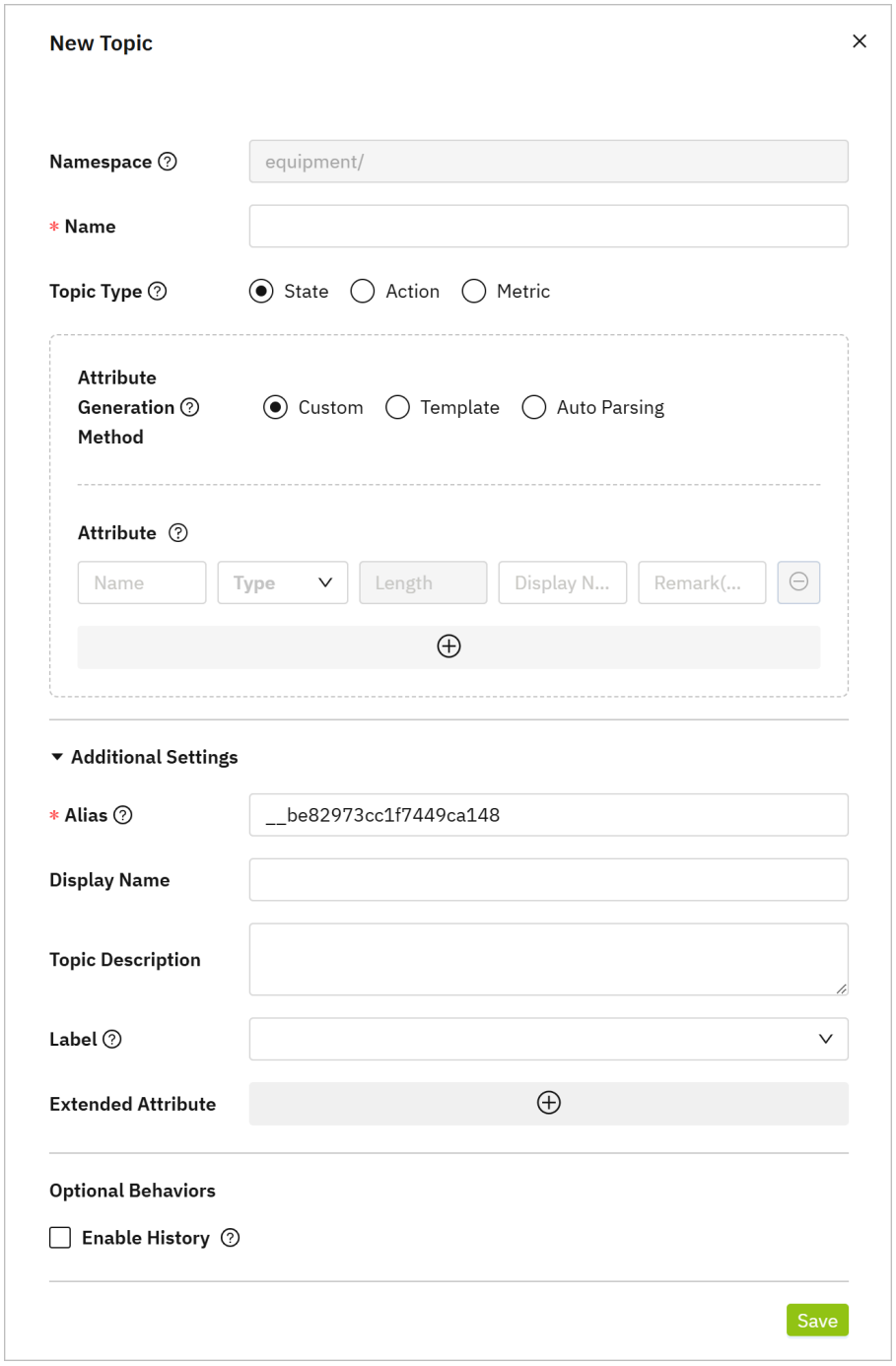
Building Models Manually

> ⓘ **note**
>
> **factory/equipment/CNC** will be used as an example, in which `factory` and `equipment` are paths and `CNC` is a topic.

1. Log in to Tier0, landed on the **Namespace** page.
2. Under **Topic**, click ＋ > **New Path** to add a path (e.g. `factory`).



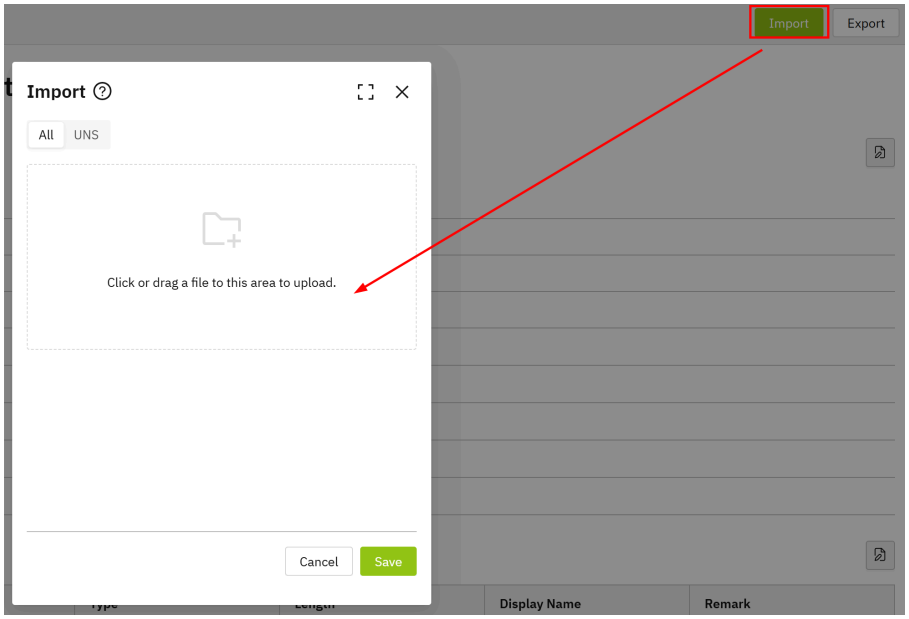
3. Enter the path information, set the alias as needed, and then click **Save**.
4. Select **equipment**, and then click ＋ > **New Topic** to add a topic (e.g. `CNC`) under it.



5. Enter the information of the topic, and then click **Save**.

Importing Models

1. Log in to Tier0, landed on the **Namespace** page.
2. Click **Import** at the upper-right corner.



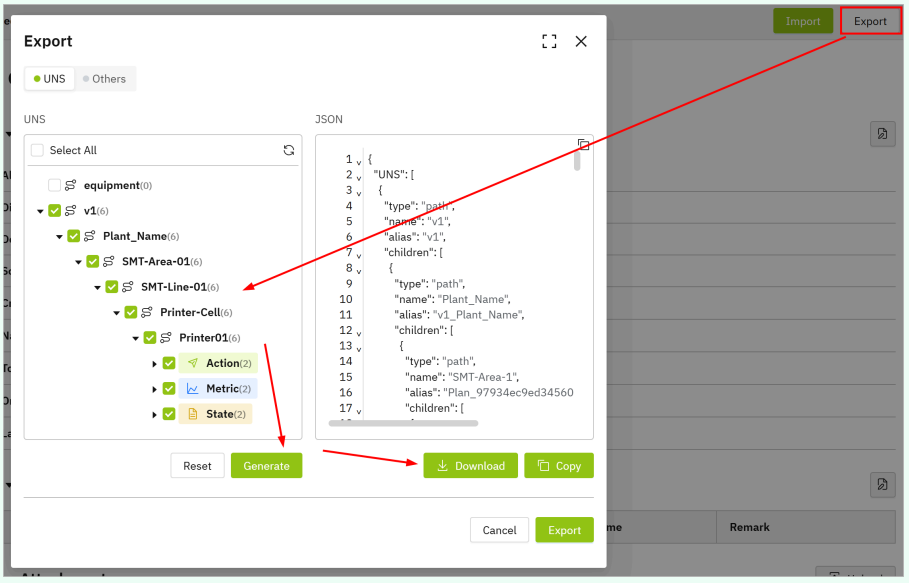
3. Import JSON to create models.

- Click **UNS**, and directly enter JSON.

- Click **Download Template** at the **UNS** tab, enter the model content and upload it at the **All** tab.

```
{
  "notes": "type:folder|file,topicType:STATE|ACTION|METRIC,dataType:TEMPLATE_TYPE|TIME_SEQUENCE_TYPE|RELATION_
  "Label": [{"name": "debug"},{"name": "dev"}],
  "Template": [],
  "UNS": [
    {
      "name": "v1",
      "type": "path",
      "children": [
        {
          "name": "Plant_Name",
          "type": "path",
          "children": [
            {
              "name": "SMT-Area-1",
              "type": "path",
              "children": [
                {
                  "name": "SMT-Line-1",
                  "type": "path",
                  "children": [
                    {
                      "name": "Printer-Cell",
                      "type": "path",
                      "children": [
                        {
                          "name": "Printer01",
                          "type": "path",
                          "children": [
                            {
                              "name": "State",
                              "type": "path",
```


> ✓ **tip**
>
> You can manually add a path and topic, export it and use it as an example for import.



4. Click **Save**.

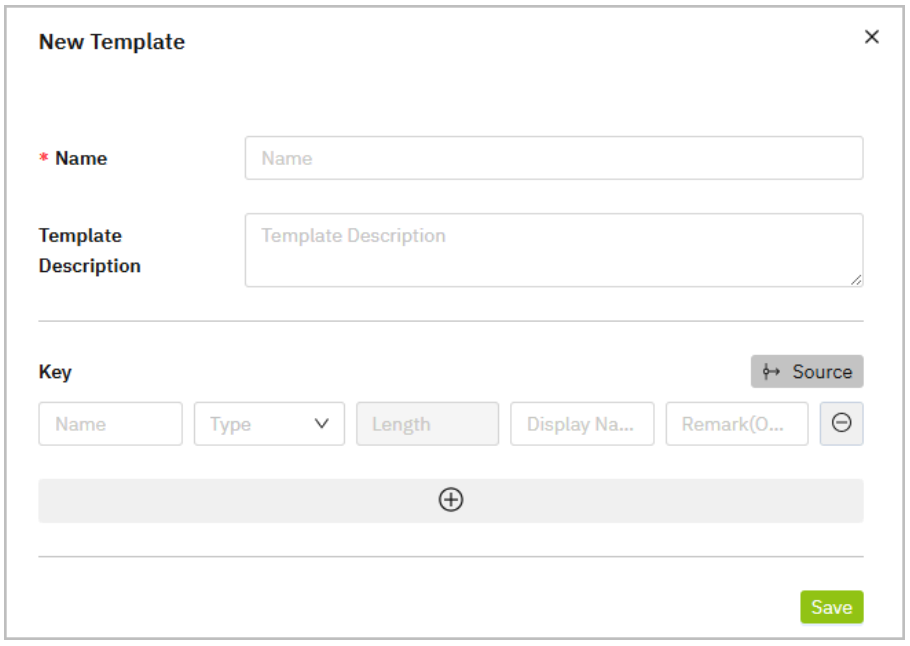
Optional Features

Building Template

1. On the **Namespace** page, click **Template**.

2. Click , and then enter the information of the template.

> ✓ **tip**
>
> Click **Source** to select added models and inherit their attributes.

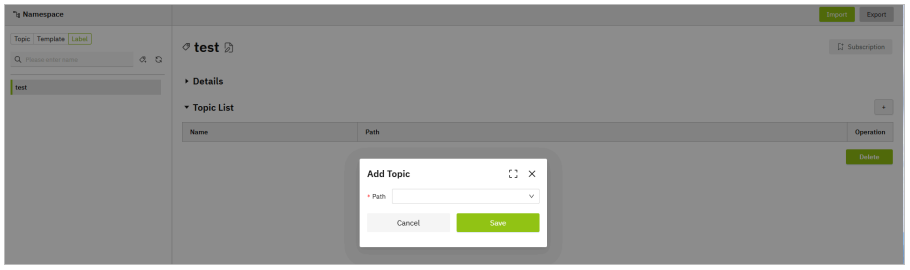


3. Click **Save**.

Creating Label

Labels are used for categorizing data models.

1. On the **Namespace** page, click **Label**.
2. Click , and then enter the label name.
3. Click **Save**, and then click + to add topics under the label.

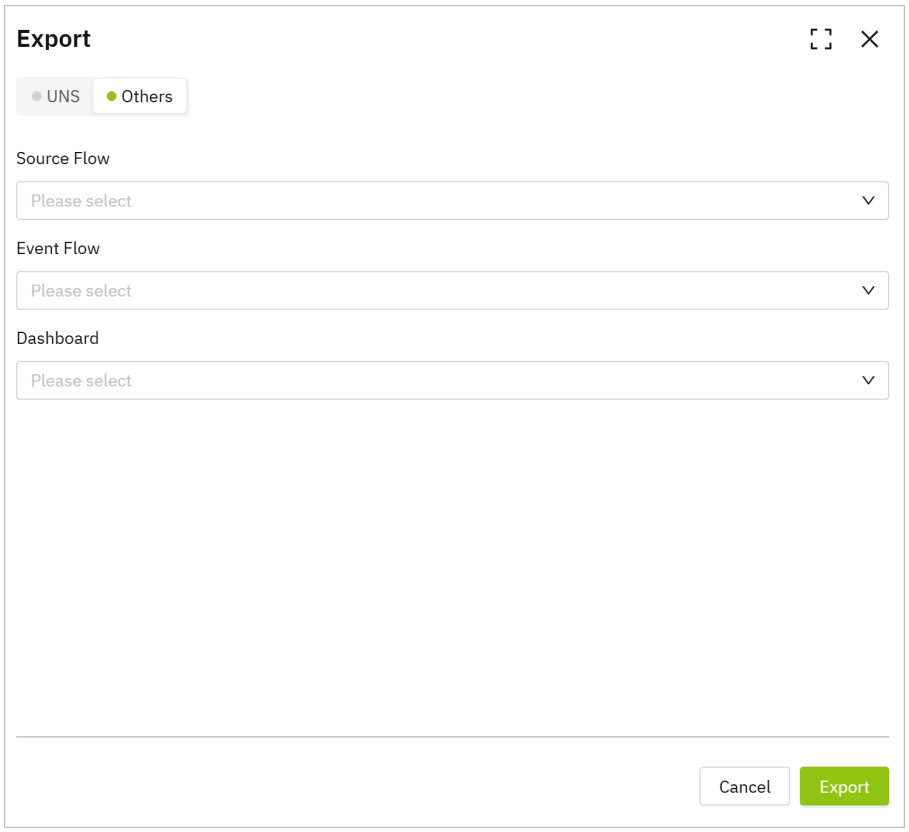


4. Click **Save**.

Exporting Data

Tier0 can export UNS models and other data, such as source flows, event flows, and data dashboards.

1. On the **Namespace** page, click **Export** at the upper-right corner.
2. Click the **Others** tab and select the corresponding data type.



3. Click **Export**.

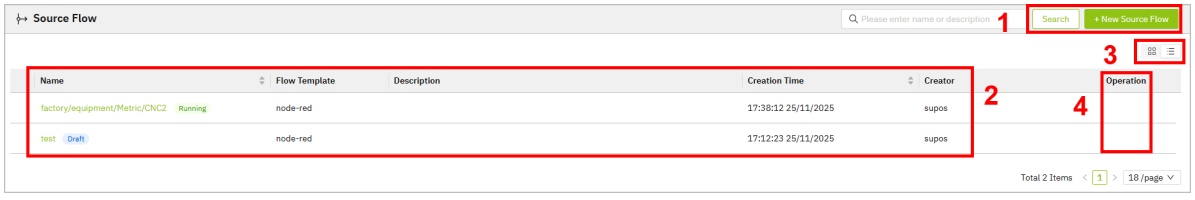
Connect Data Sources

How to Add Data Sources

Once the data model is built under **Namespace**, you need to connect real data to the model for subsequent processing and use.

Adding Data Flow Manually

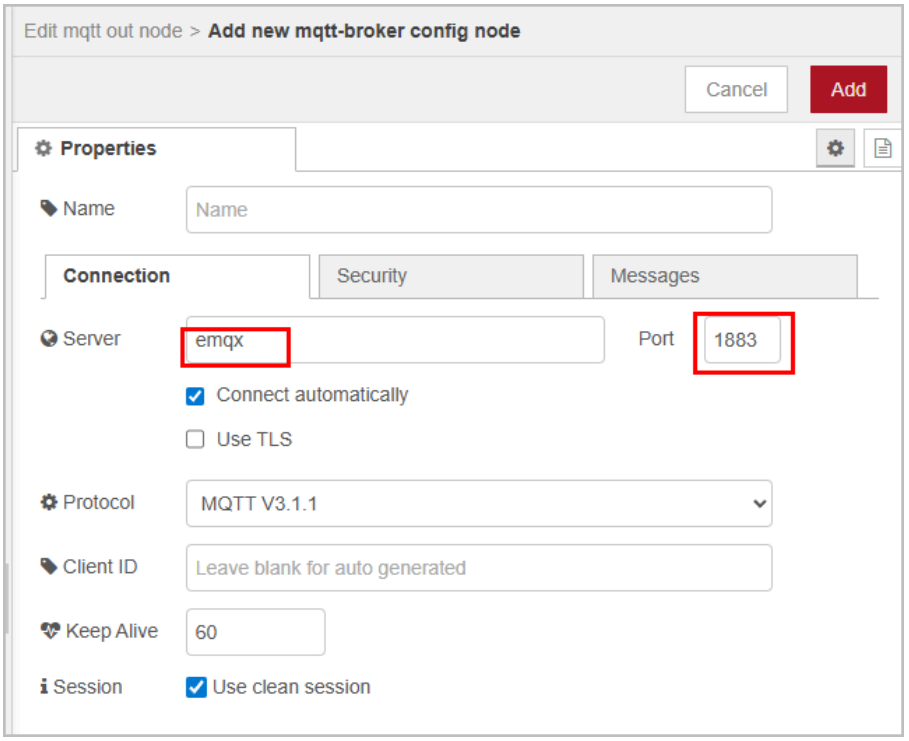
1. Log in to Tier0, go to **UNS** > **Source Flow**.



Index	Item	Description
1	Search	Search for existing data flows.
	New Source Flow	Add a data flow.
2	Flow List	Displays flow information. Click the flow name to start editing the data flow.
3	Display	Switch flow display between blocks and list.
4	Flow Operations	Copy / Edit / Delete the data flow.

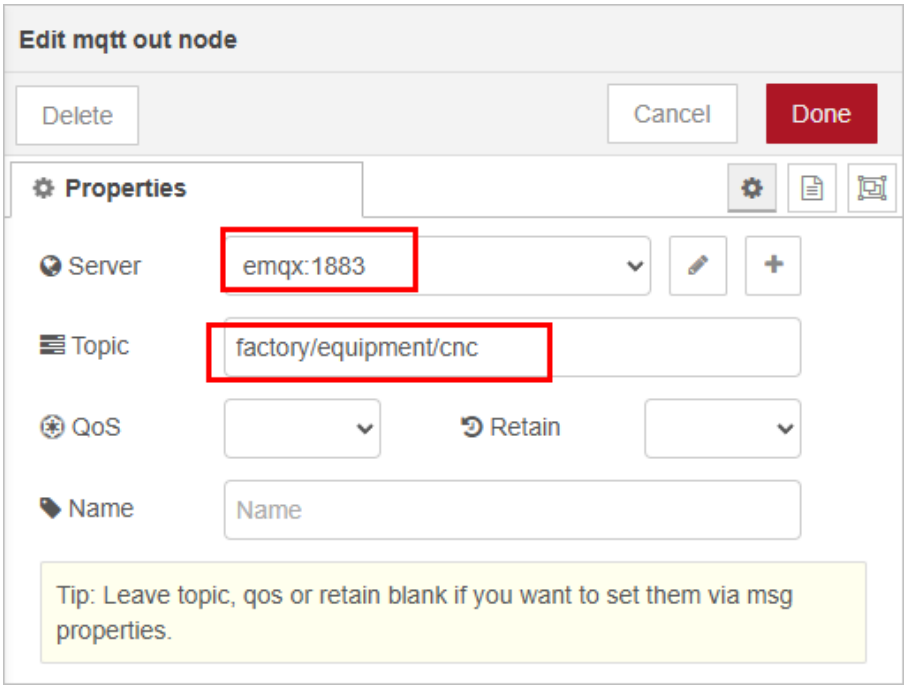
2. Click **New Source Flow**, enter a name and click **Save**.
3. Click the flow you just added, and drag in nodes according to your data source type (e.g. `modbus`).
4. Drag in an `mqtt out` node, and finish its configurations.

    1. Add the internal broker via `emqx:1883` .



    2. Set its subscription topic to the data model in **Namespace**.

        For example, the data model structure is `factory/equipment/cnc` , the topic will be `factory/equipment/cnc` .

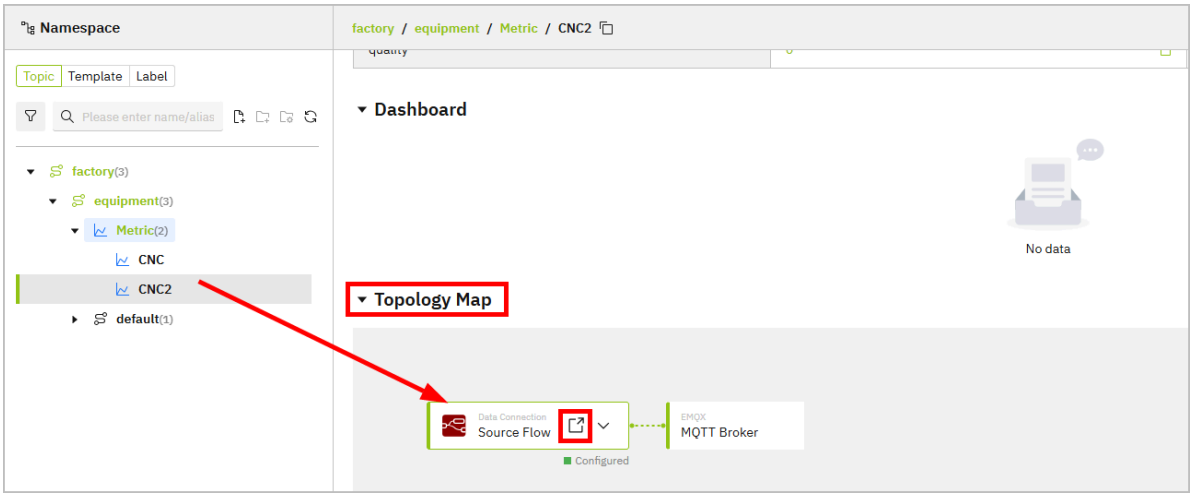


5. Click **Deploy** at the upper-right corner, trigger the flow and the data will be sent to corresponding model under **Namespace**.

Configuring Data Source through Generated Flow

> ⓘ **note**
>
> Data flow will only be automatically generated when you select **Mock Data** while creating topics.

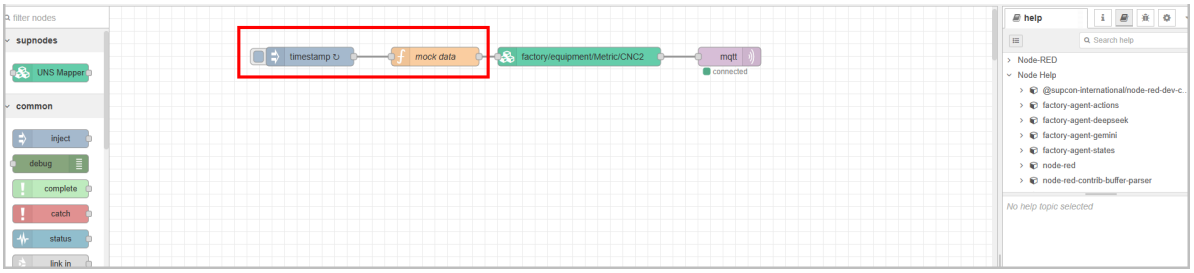
1. Log in to Tier0, landed on the **Namespace** page, and under the **Topic** tab, select a file.
2. Scroll down to **Topology Map**, click the icon on **Source Flow** to redirect to the generated data flow under **Source Flow**.



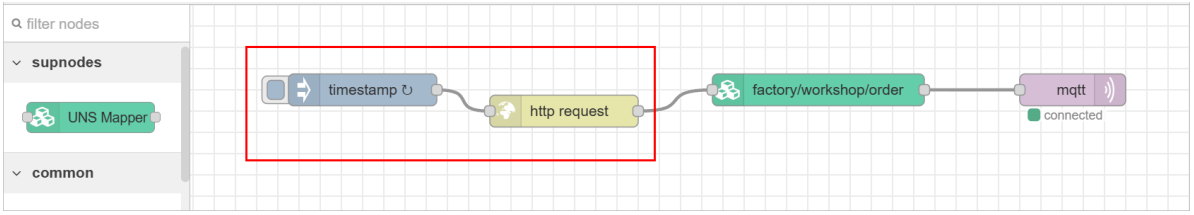
> ✓ **tip**
>
> Click ⌄ to change the bound data flow.

3. Change the data source of the generated flow.

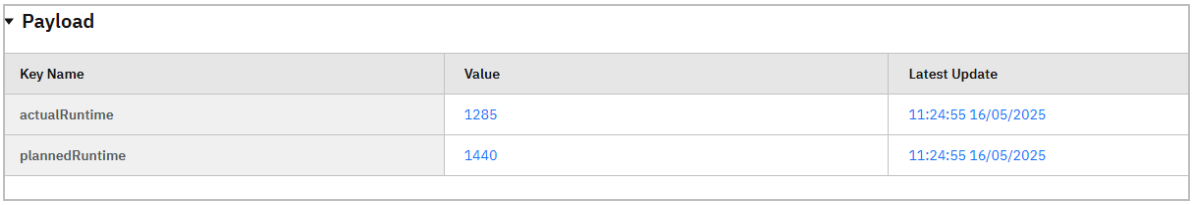
> ⓘ **info**
>
> Make sure the returned data of the source node is **JSON object**. Currently, Tier0 can only parse this data type.



4. Delete the `mock data` function node and connect the source node to the rest of the flow.



5. Click **Deploy** at the upper-right corner, and then trigger the flow.
6. Go back to **Namespace**, check whether the data is received.

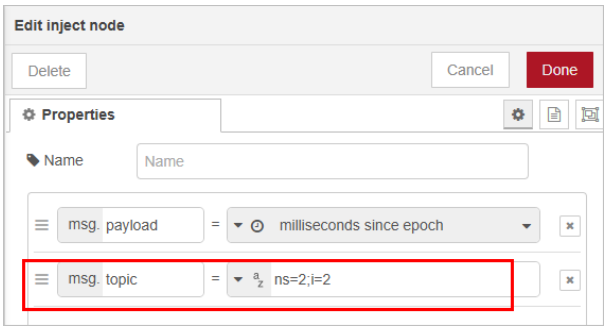


Common Data Sources

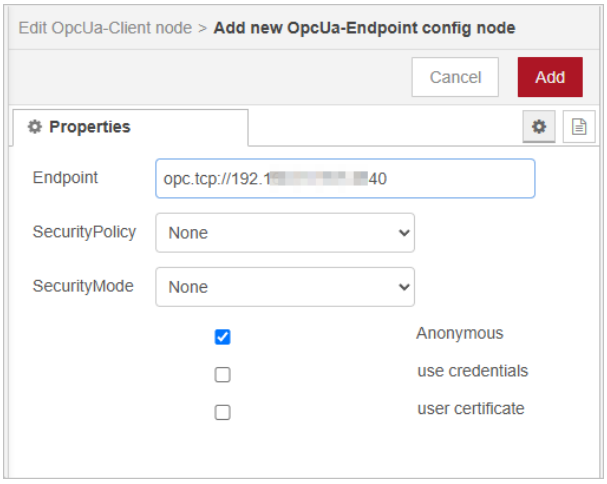
Connecting OPC UA

Read Data from OPC UA Server

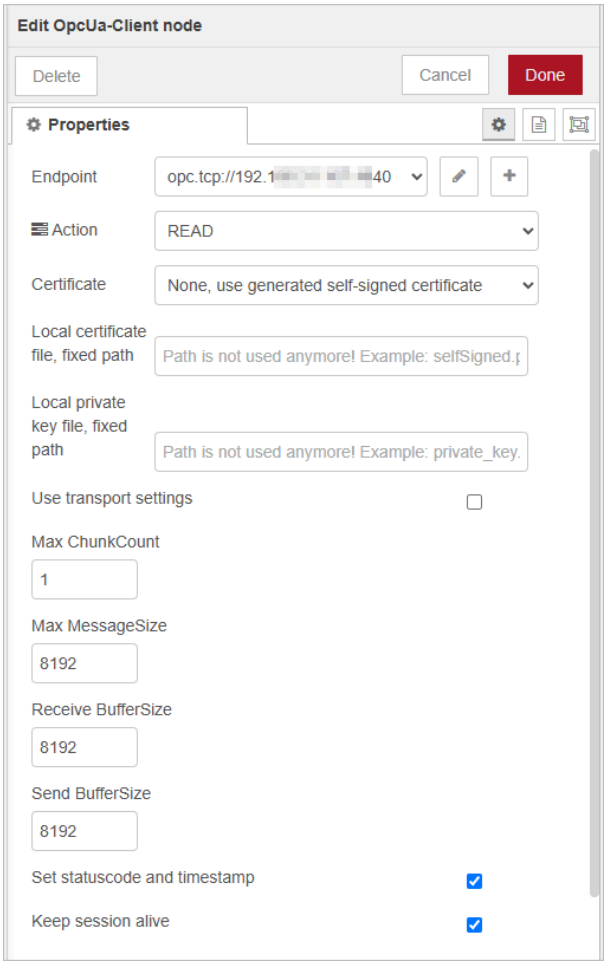
1. In **Source Flow**, drag an **inject** node.
2. Double-click the node, and then enter the **NodeId** from your OPC UA server.



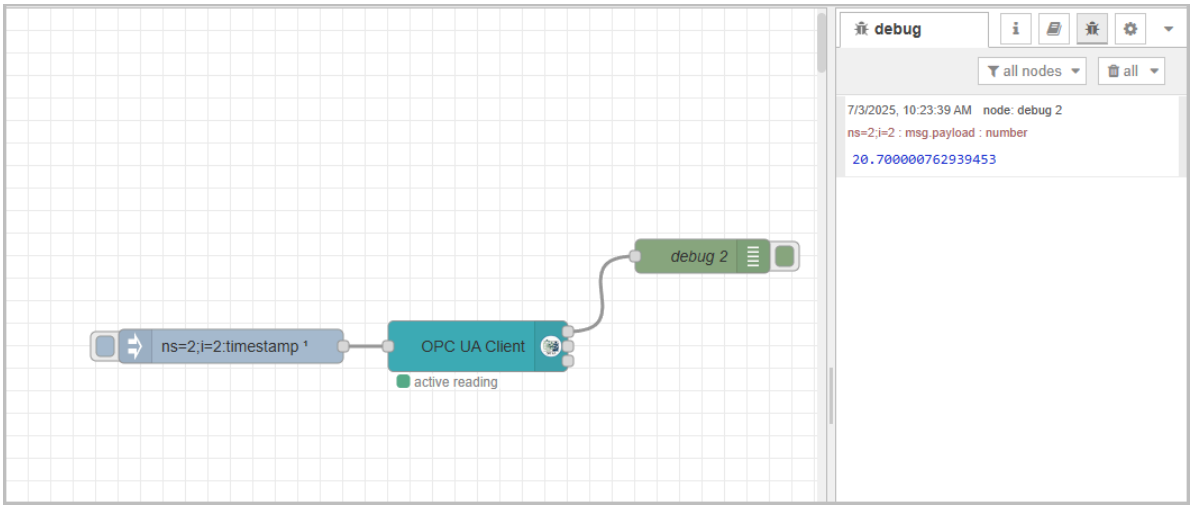
3. Drag an **OpcUa-Client** node, and then double-click it.
4. Click ✚ next to **Endpoint** to enter the information of your OPC UA server.



5. Click **Add**, and set **Action** to **READ**.
6. Leave everything else as default, and then click **Done**.



7. Connect the **inject** node to the **OpcUa-Client** node, and then connect a debug node at the end.
8. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.



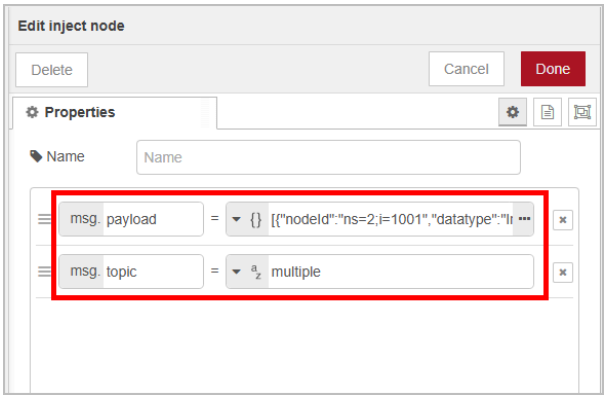
Subscribing Data from OPC UA Server

1. In **Source Flow**, drag an **inject** node.
2. Double-click the node, and then enter the **NodeId** from your OPC UA server.

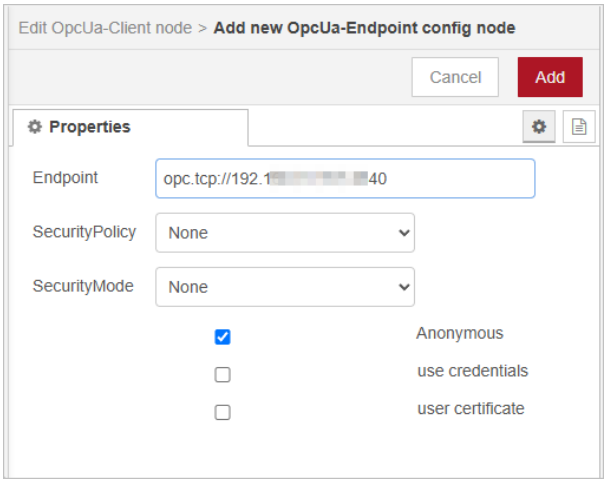
- msg.payload:

```
[
    {
        "nodeId": "ns=2;i=1001",
        "datatype": "Int32"
    },
    {
        "nodeId": "ns=2;i=1002",
        "datatype": "Int32"
    }
]
```

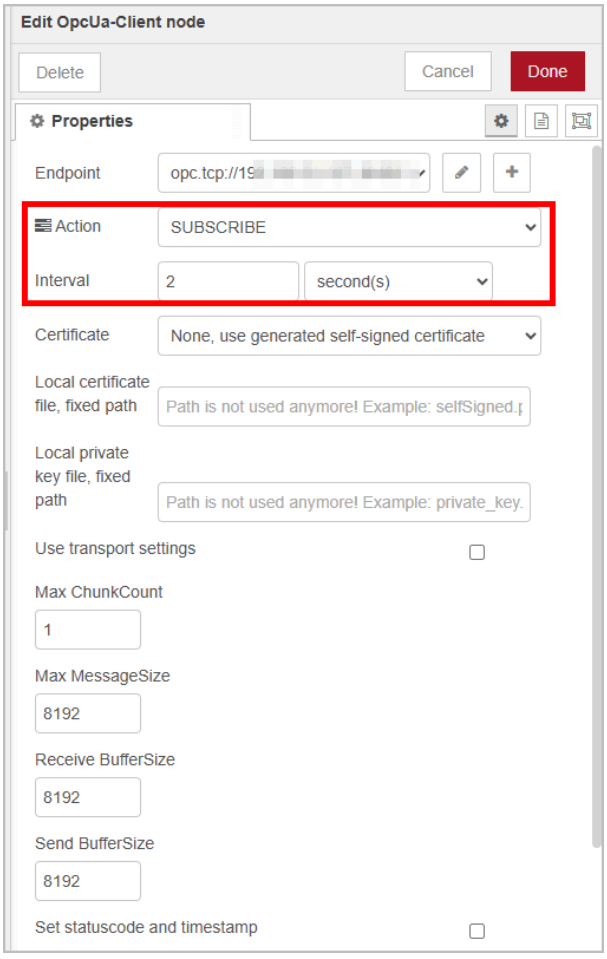
- msg.topic: multiple



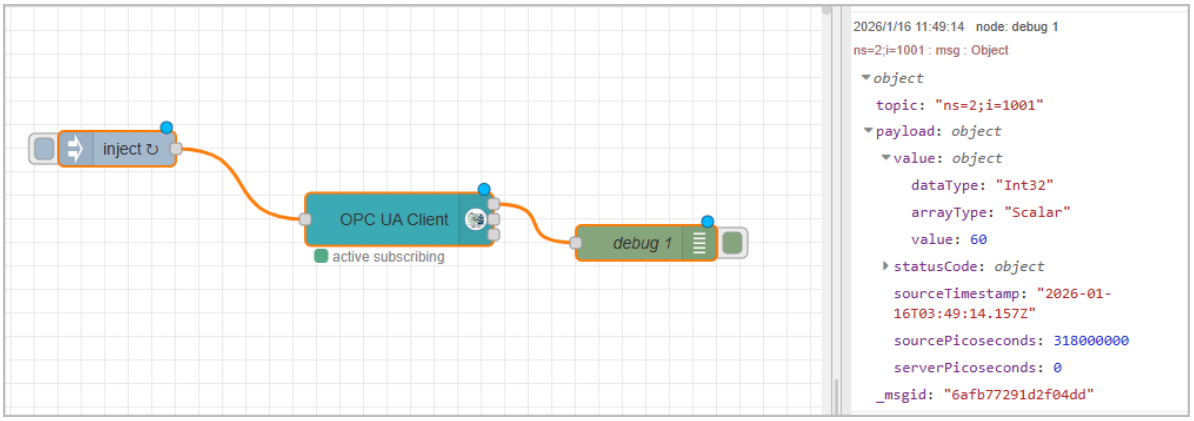
3. Drag an **OpcUa-Client** node, and then double-click it.
4. Click ➕ next to **Endpoint** to enter the information of your OPC UA server.



5. Click **Add**, and set **Action** to **SUBSCRIBE**.
6. Set the subscription interval, and leave everything else as default, and then click **Done**.

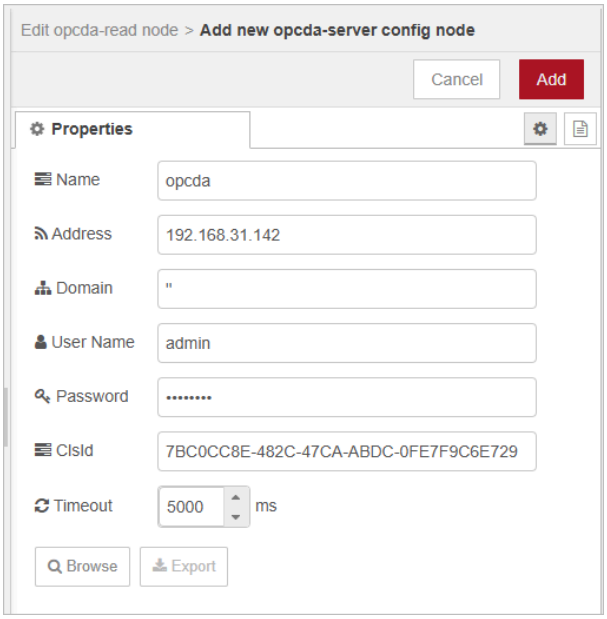


7. Connect the **inject** node to the **OpcUa-Client** node, and then connect a debug node at the end.
8. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.



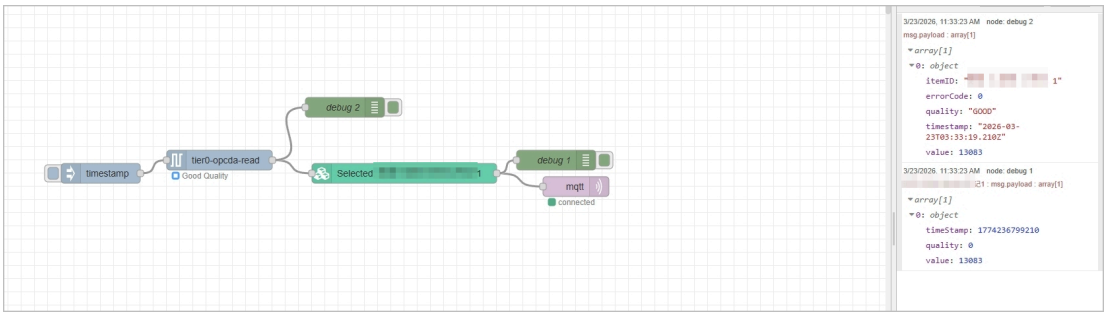
Connecting OPC DA

1. In **Source Flow**, drag an **inject** node.
2. Drag an **opcda-read** node, and then double-click it.
3. Click ✚ next to **Server** to enter the information of your OPC DA server.



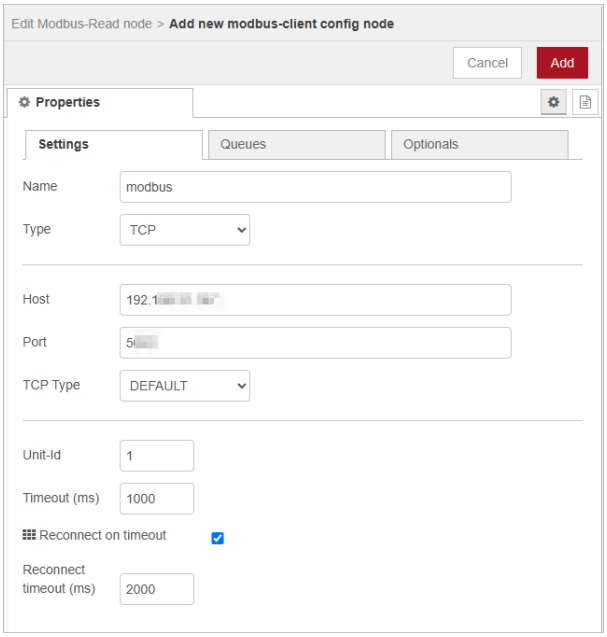
Parameter	Description
Address	OPC DA server IP address.
Domain	The domain name of your OPC DA server.
User Name/Password	The username and password you use to log in to the DA server.
ClsId	The application ID of the DA connetion service you use. For example, KepServer.

5. Click **Browse** to see the connected channels.
6. Click **Add**, and then under **Items**, add the items needed.
7. Connect the **inject** node to the **opcda-read** node, and then connect a debug node at the end.
8. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.

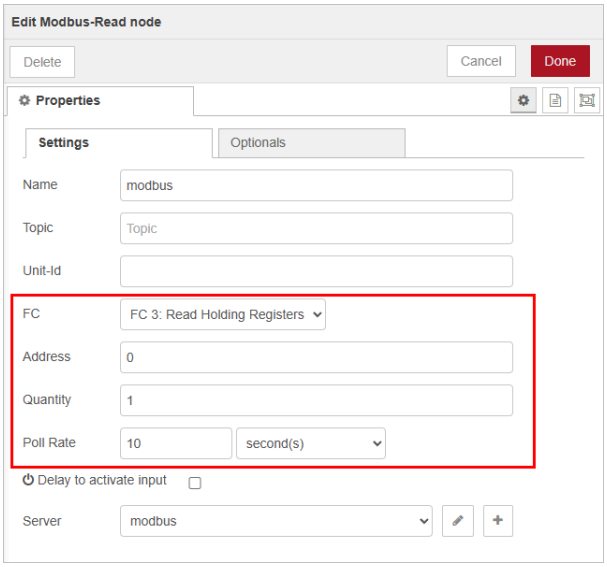


Connecting Modbus

1. In **Source Flow**, drag a **Modbus Read** node.
2. Double-click the node, and then click ✚ next to **Server** to enter the information of your modbus server.

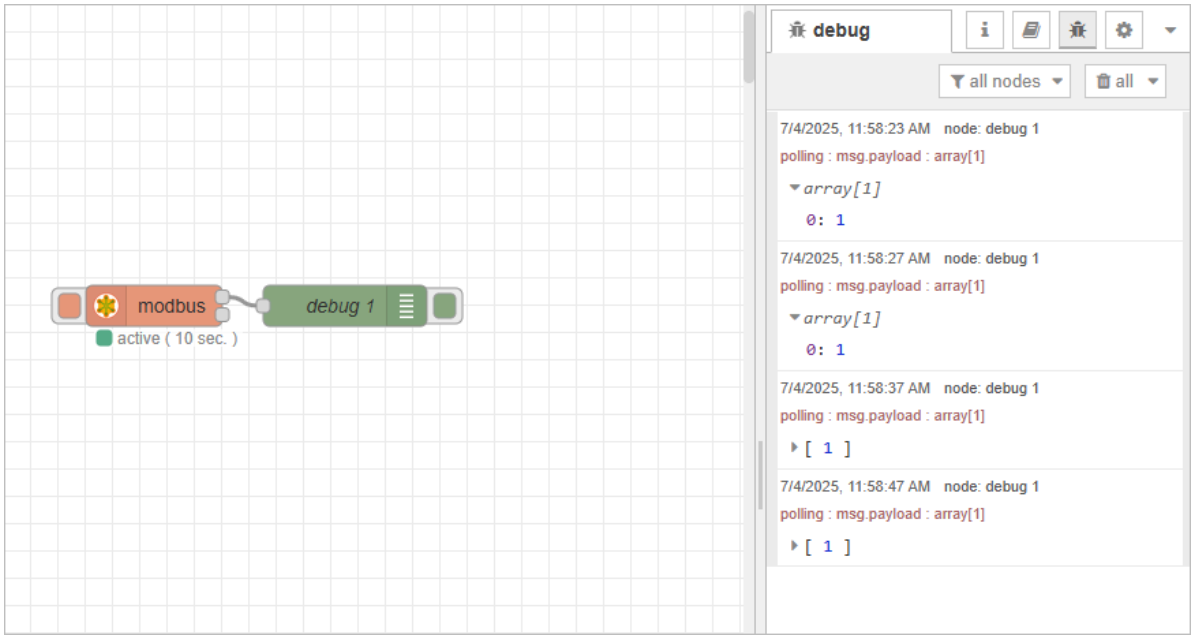


3. CLick **Add**, and then enter the information of the data to be connected.



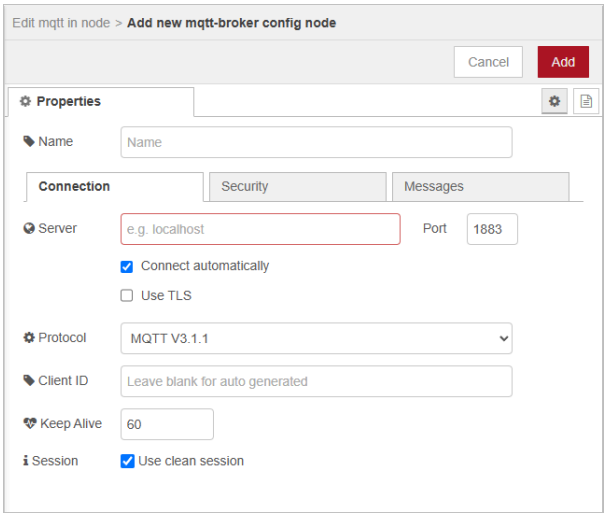
Parameter	Description
Name	The display name of the node (for visual identification only).
Topic	Optional topic string passed to output messages.
Unit-Id	The Modbus slave ID of the target device (typically 1–247).
FC	Function code that specifies the Modbus action, e.g., FC 3 = Read Holding Registers.
Address	The starting register address to read from (usually zero-based).
Quantity	The number of consecutive registers to read.
Poll Rate	How often the node polls the Modbus device (e.g., every 10 seconds).
Delay to activate input	Optional delay before activating the input signal for polling.
Server	Reference to a configured Modbus server (IP, port, protocol, etc.).

4. Click **Done**.
5. Connect a debug node at the end.
6. Click **Deploy** at the upper-right corner, and then click **Modbus Read** node to trigger the flow.



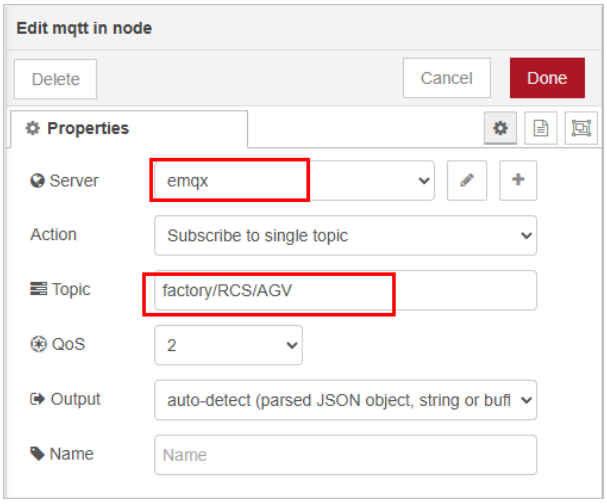
Connecting MQTT

1. In **Source Flow**, drag an **mqtt in** node.
2. Double-click the node, and then click + next to **Server**.

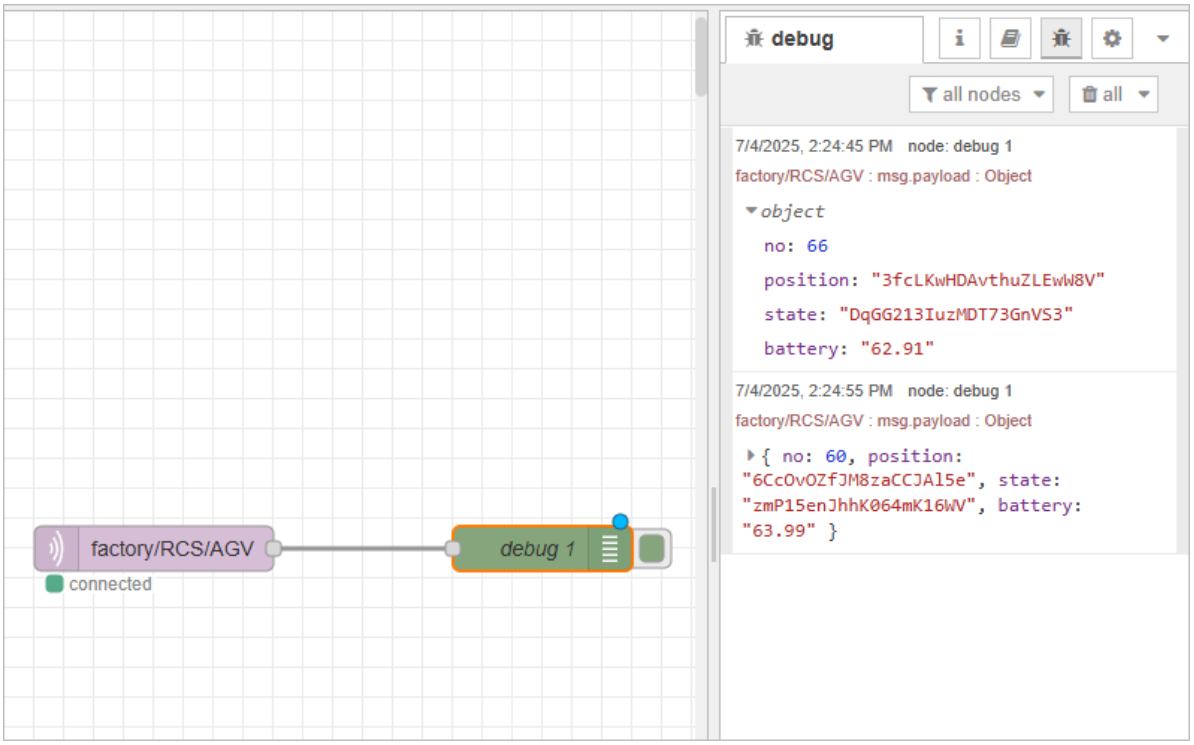


Parameter	Description
Name	Optional name for the MQTT broker configuration (for internal reference).
Server	The hostname or IP address of the MQTT broker (e.g., `localhost` or `broker.hivemq.com`).
Port	The port used to connect to the MQTT broker (default is `1883` for non-TLS, `8883` for TLS).
Connect automatically	If enabled, Node-RED will try to connect to the broker automatically when the flow starts.
Use TLS	Enable this option if the broker requires a secure TLS/SSL connection.
Protocol	The MQTT protocol version to use for connection (e.g., MQTT v3.1.1 or v5.0).
Client ID	Optional client ID. If blank, a unique one will be auto-generated. Must be unique per client.
Keep Alive	The maximum period (in seconds) between communications with the broker. Helps detect dropped connections.
Use clean session	If enabled, no persistent session is maintained with the broker; otherwise, session state (like subscriptions) is retained.
Username (Security tab)	Username for authenticating with the MQTT broker, if required.
Password (Security tab)	Password corresponding to the username for broker authentication. Stored securely.

3. Click **Add**, and then enter the topic to be subscribed.
4. Click **Done**.



5. Connect a debug node at the end.
6. Click **Deploy** at the upper-right corner, and then once the topic receives data, the flow will be triggered.



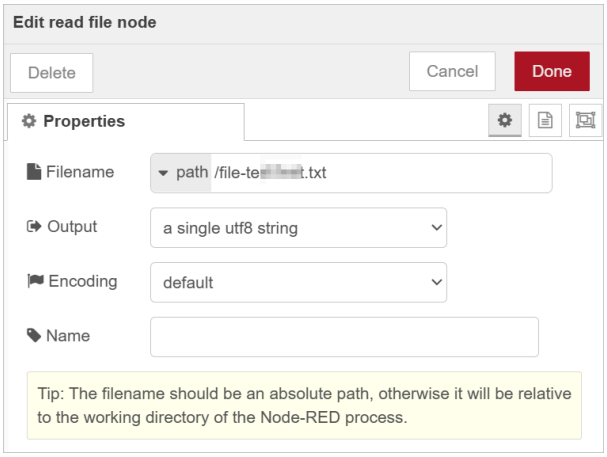
Connecting File

> ⓘ **info**
>
> Before processing files in **Source Flow**, you need to put the file in to the corresponding docker.
>
> ```
> scp C:\Users\you\Desktop\myfile.txt username@server IP:/home/username/
>  //copy the file from local to server
> docker cp /root/config.json <docker name or ID>:/app/config.json
>  //copy the file from server to docker
> ```

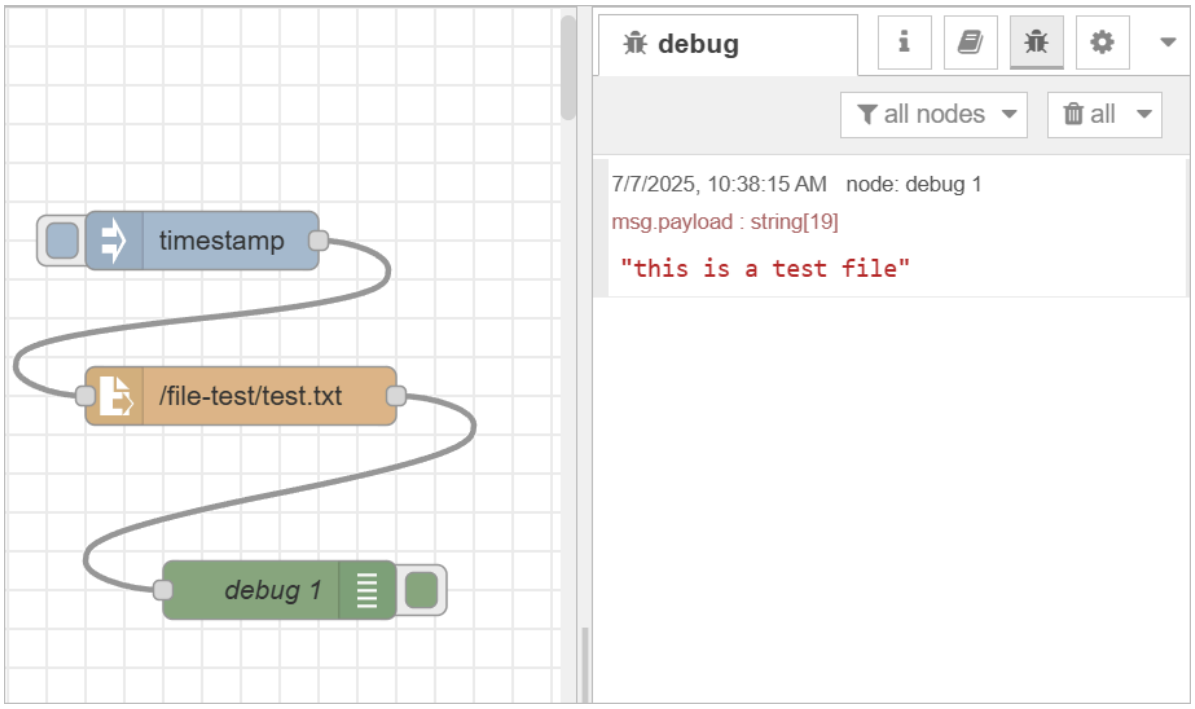
> ✓ **general docker commands**
>
> | Command | Description |
> | --- | --- |
> | `docker ps` | Lists all running Docker containers. |
> | `docker ps \| grep [container]` | Filters running containers based on the specified docker information. |
> | `docker exec -it [container] /bin/sh` | Starts a shell session inside a running container using `/bin/sh`. |
> | `mkdir [directory]` | Creates a new directory with the specified name inside a container. |

Connecting a Text File

1. In **Source Flow**, drag an **inject** node.
2. Drag a **read file** node, and then double-click it.
3. Enter the file path and output configuration.

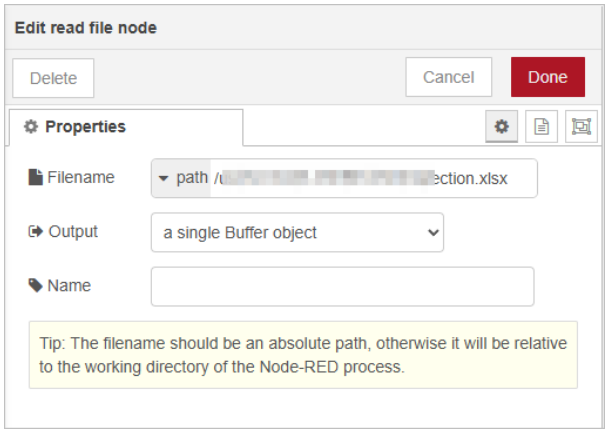


4. Click **Done**, and then connect a debug node at the end.
5. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.

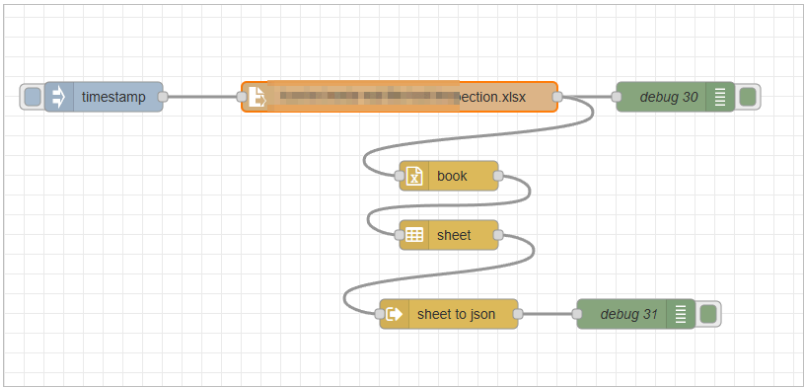


Connecting an Excel File

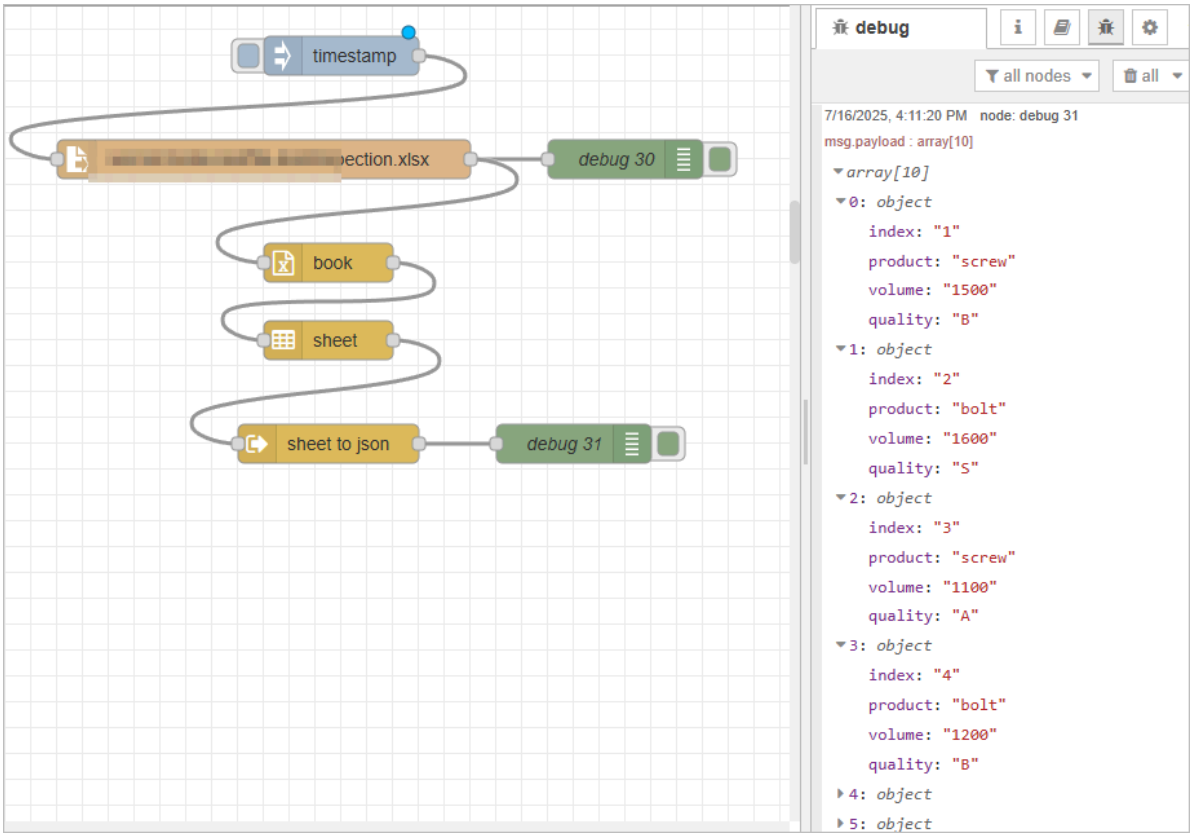
1. In **Source Flow**, drag an **inject** node.
2. Drag a **read file** node, and then double-click it.
3. Enter the file path and output configuration.



4. Click **Done**, and then connect a **book**, a **sheet** and a **sheet to json** node by order to convert table data to json.



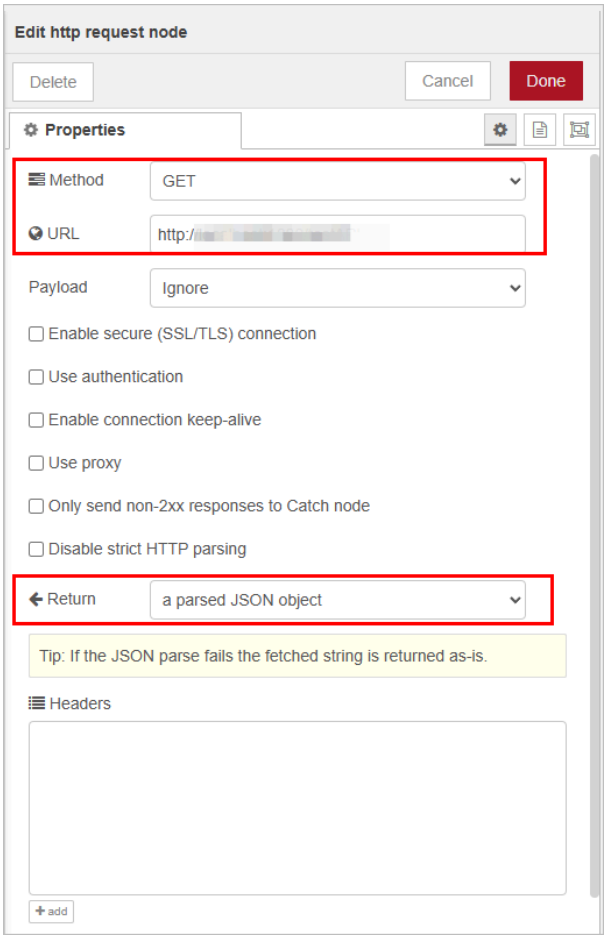
5. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.



Connecting RestAPI

1. In **Source Flow**, drag an **inject** node.
2. Drag an **http request** node, and then double-click it.
3. Enter the API information.

> ✓ **tip**
>
> Set the **Return** to **a parsed JSON obbject** for Tier0 namespace to parse.



5. Click **Done**, and then connect a debug node at the end.
6. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.